PENETRATION TEST REPORT · V1.2

Security Assessment of the OSCE Guide Platform

A white-box security assessment of the new stack — the FastAPI backend, the Next.js web application, and the LiveKit voice workers — conducted ahead of production cutover and institutional due diligence.

ASSESSMENT TYPE	DATES	OVERALL RISK	FINDINGS
Internal white-box (local)	24–25 Sep 2026 (incl. deep-dive verification round)	Medium	20 (0 Critical)
STANDARDS	SCORING		
OWASP WSTG · ASVS L2 · API/LLM Top 10	CVSS 3.1		

01 Executive summary

The platform's core security architecture is sound. No Critical issues and no unauthenticated remote code execution were found. The two most serious application-level flaws were found and fixed during the engagement, each with a regression test.

0	3	8	7	2
CRITICAL	HIGH	MEDIUM	LOW	INFO
	2 fixed	4 fixed	5 fixed · 1 mitig.	

11
RESOLVED
fixed w/ tests

The assessment was white-box (full source access) and mapped to the standards an accredited third-party firm would use: OWASP WSTG, ASVS Level 2, the API and LLM Top 10s, and the CIS benchmarks, scored with CVSS 3.1. Every confirmed issue was captured as an automated regression test so that fixes are provable and durable. A second, deep-dive **verification round** (25 Sep) then swept the classes an automated scanner alone would miss — SSRF, insecure deserialization, template and command injection, XXE, ReDoS, mass-assignment, BFLA, and the payment and voice-token authorization paths. It cleared those classes as *absent* and surfaced six further findings (SEC-015 to SEC-020), four of which were fixed with regression tests; it also corrected SEC-011 down to *mitigated* after confirming the code already blocks it. A subsequent **remediation pass** (25 Sep) then cleared the backlog of every finding that a code change could safely resolve — the voice session-length cap (SEC-008), the platform-api container hardening (SEC-009), the worker pod securityContext and NetworkPolicy (SEC-010), the circuit-enrolment roster lock-down (SEC-014), and the Entra SSO `state` / `nonce` and stream-ticket single-use controls (SEC-015, SEC-016) — each with its own regression test. Eleven of the twenty findings are now fixed; what remains open is rate-limit hardening (SEC-003/004), assistant `extras` validation (SEC-007), the enforcing CSP (SEC-006, deferred to staging), owner-only secret rotation (SEC-012), and three low/informational items.

Top issues for leadership

1. **Privilege escalation between organisation roles (High — fixed).** A “partner admin” — a role meant for a practice surface that must never run an examination — could promote itself or any member to `institution_admin` or `owner`, the roles that sign off candidate results, because the role field accepted any string. In an examination product this crosses the integrity boundary that matters most. Now validated against an allow-list, with a granter unable to grant above its own tier.
2. **Cross-context script execution via unsanitised content (High — fixed).** The web app rendered markdown (including raw HTML) straight into the page with no sanitiser. Its inputs include station text authored by organisation staff, help articles, and **the AI tutor's own chat replies** — so a lower-trust author, or a manipulated model, could run JavaScript in another user's browser. A sanitiser now strips scripts, event handlers and dangerous URLs.
3. **Secrets in repository history and on disk (High — owner action).** Backend git history contains private keys, an access token is embedded in a git remote URL, and a Google Cloud service-account key sits in the working tree. These must be **rotated**. This is the one High item that needs an owner decision rather than a code fix.

The remaining Medium and Low items are hardening: rate-limiting robustness, an enforcing browser Content-Security-Policy, LLM cost/abuse caps, container and Kubernetes posture,

and an SSO-only account edge case. None is independently exploitable for a full compromise.

> **Bottom line for an auditor:** the platform is built on sound foundations, the two most serious application flaws were found and fixed with tests during the assessment, and a prioritised roadmap covers the rest.

02 Scope & method

Round one: a local, white-box assessment of the new stack before it carries production traffic. Production is a separate follow-on engagement (v2, passive checks).

COMPONENT	DESCRIPTION	COMMIT
platform-api	FastAPI backend · ~370 routes / ~59 routers · JWT auth · PostgreSQL	**b68dc7f**
platform-web	Next.js 16 · BFF refresh cookie + in-memory access token	**6b62e61**
assessment-platform	LiveKit assessment + assistant voice workers on K3s	**39692ae**

Method. White-box code review with targeted dynamic tests against an *isolated* local instance — a dedicated database, dummy secrets, a separate port. The shared dev stack was untouched. Route inventory was taken from `/openapi.json`.

Out of scope. LiveKit Cloud SFU, Stripe infrastructure (test mode only; webhook verification in scope), the production environment (v2), DoS/load testing, and the retired legacy apps (noted only where their artefacts affect the new repos).

03 Findings summary

Twenty findings. Severity is a CVSS 3.1 base score adjusted for reachability in this system.

ID	TITLE	SEV	CVSS	COMPONENT	STATUS
SEC-001	Org-role privilege escalation via unvalidated role	High	8.1	platform-api	Fixed

ID	TITLE	SEV	CVSS	COMPONENT	STATUS
SEC-002	Stored / model-driven XSS via unsanitised markdown	High	7.6	platform-web	Fixed
SEC-012	Secrets in git history & working tree	High	7.5	repos	Owner
SEC-003	Rate-limit key spoofable (XFF) and fails open	Medium	5.3	platform-api	Backlog
SEC-004	No rate limit on registration / growth endpoints	Medium	5.3	platform-api	Backlog
SEC-006	No enforcing Content-Security-Policy on web app	Medium	4.7	platform-web	Partial
SEC-007	Unvalidated assistant `extras` → prompt steering & cost	Medium	5.4	platform-api	Backlog
SEC-008	No cap on voice session length → cost abuse	Medium	5.3	assessment	Fixed
SEC-009	Container runs as root; build-ARG token in history	Medium	5.9	platform-api	Fixed
SEC-015	Entra SSO: no OAuth `state` / OIDC `nonce`	Medium	5.4	platform-api	Fixed
SEC-018	Slide renderer path traversal via LLM image URLs	Medium	5.0	platform-api	Fixed
SEC-005	Missing security headers; schema exposed when deployed	Low	3.7	platform-api	Fixed
SEC-019	Clerk session id spliced unescaped into API URL	Low	3.1	platform-api	Fixed
SEC-010	Worker pods lack securityContext / NetworkPolicy	Low	4.0	assessment	Fixed
SEC-013	Photo upload validates extension only (no content sniff)	Low	3.1	platform-api	Backlog
SEC-014	Org may enrol an un-rostered arbitrary user id	Low	3.5	platform-api	Fixed
SEC-016	Stream/voice ticket time-bound but not single-use	Low	2.6	platform-api	Fixed

ID	TITLE	SEV	CVSS	COMPONENT	STATUS
SEC-011	Password reset on SSO-only accounts	Low	~2.0	platform-api	Mitigated
SEC-017	Stripe checkout omits `metadata.price_id` (reliability)	Info	–	platform-api	Backlog
SEC-020	Community `set_like` skips audience check	Info	–	platform-api	Backlog

04 Detailed findings

Each finding gives its CVSS vector, location, impact, evidence and remediation. Fixed findings carry a passing regression test.

SEC-001 Organisation-role privilege escalation via unvalidated `role`

Fixed

CVSS 3.1 8.1 · AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N | **OWASP** API5:2023 · CWE-269

Membership create/update routes are gated by `OrgAdmin`, which correctly admits `partner_admin` — but the `role` on the request body was an unconstrained string written straight to the database. The tenancy domain deliberately separates who may administer an org from who may sign off a candidate's result, and `partner_admin` is excluded from the latter “by design.” Because the role was never checked, a partner admin could set its own role — or a new member's — to `institution_admin` or `owner`, or to any undefined string.

IMPACT

Vertical privilege escalation to examination-signing authority within an organisation — an integrity issue of the first order in an assessment product.

EVIDENCE

`tests/security/test_membership_role_escalation.py`: a partner admin's grant of `institution_admin` returned `201` before the fix (must be `403`); an undefined role returned `201` (must be `422`). Both fail on the vulnerable code and pass after the fix.

REMEDIATION (APPLIED)

Added a role allow-list and rank to `tenancy/domain.py` (`GRANTABLE_ROLES`, `may_grant_role`), and a guard in `service_memberships.py` that runs before every

membership write: an unknown role is `422`, a known role above the granter's tier is `403`. Covers create, update and bulk add.

SEC-002 Stored / model-driven XSS via unsanitised markdown

Fixed

CVSS 3.1 7.6 · AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N | **OWASP** LLM02:2025 · A03:2021 · CWE-79

The renderer ran `rehype-raw` (raw HTML parsed) then wrote the result straight into the DOM with `dangerouslySetInnerHTML` and **no sanitiser**. Any `<script>`, `onerror` handler or `javascript:` URL in the source executed in the viewer's session. Its inputs are lower-trust: ortho station content authored by org staff, help articles, and — most importantly — **the AI tutor's own chat replies**, which a candidate can steer via prompt injection. The error fallback also injected raw content verbatim.

> This is an *insecure LLM output handling* issue as well as classic stored XSS: content the model emits was rendered as live HTML.

EVIDENCE

`platform-web/src/foundation/ui/renderMarkdown.test.ts` — 7 tests confirm `<script>`, `onerror`, `javascript:` links and `<iframe>` are stripped while ordinary formatting, safe links and code-block classes survive.

REMEDIATION (APPLIED)

Added `rehype-sanitize` after `rehype-raw` with rehype's GitHub-derived default schema (no scripts, handlers, dangerous URLs, iframes). Extracted the pipeline into a unit-tested module and changed the error fallback to render *escaped text*, never raw HTML.

SEC-012 Secrets in git history and working tree

Owner action

CVSS 3.1 7.5 · AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N | **OWASP** A07:2021 · CWE-798

The secret scan and manual review found: an access token embedded in the `backend/.git/config` remote URL; TLS private-key files (`cloudflaressl.key`, `cloudflaressl.pem`, `osceguide.pem`) in backend git history; and a Google Cloud service-account key (`serviceAccountKey.json`, containing a `private_key`) in the working tree at the repo root. (Web/legacy scan hits were reviewed and are variable-name false positives.)

IMPACT

If any secret is still valid: repository push access, TLS-certificate impersonation, or GCP access. Deleting the files is not sufficient — history and prior clones retain them.

REMEDIATION (OWNER DECISION)

- **Rotate** every secret: reissue the token, re-key the TLS certificates, create a new GCP key and disable the old.
- Move `serviceAccountKey.json` out of the tree into a secret manager; git-ignore it everywhere.
- Replace the tokenised git remote with SSH or a credential helper.
- Consider a history rewrite after rotation; enable push-protection / secret-scanning on the remote.

> Left open by design: this is a credential-rotation decision for the owner, not a change the assessment should make unilaterally.

SEC-003 Rate-limit key is spoofable and the limiter fails open

Backlog

CVSS 3.1 5.3 · AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N | **OWASP** API4:2023 · CWE-307

`core/rate_limit.py` takes the caller key from the first `X-Forwarded-For` value (spoofable by a direct caller, giving a fresh bucket per request), and lets requests through when Redis is unavailable (fails open). This weakens brute-force protection on login and password reset. The code comment is candid that it “raises the cost of an attack rather than preventing one.”

REMEDIATION

Trust `X-Forwarded-For` only for the known proxy hop; for auth-specific limits, fail closed when the store is unavailable, with a tight allowance. Confirm the edge proxy strips inbound `X-Forwarded-For` (verify in v2).

SEC-004 No rate limit on registration / growth endpoints

Backlog

CVSS 3.1 5.3 · AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:L | **OWASP** API4:2023

Registration and some growth endpoints have no rate limit — confirmed live: repeated registrations succeeded without throttling, each triggering a transactional email. This enables mass account creation and email-send abuse.

REMEDIATION

Per-IP and per-email limits on registration and email-triggering endpoints; consider a CAPTCHA or proof-of-work on public sign-up.

SEC-007 Unvalidated assistant `extras` → prompt steering and cost

Backlog

CVSS 3.1 5.4 · AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:L/A:N | **OWASP** LLM01:2025

`POST /assistant/sessions` accepts a free-form `extras` object used to build the consulting-doctor system prompt (`doctor_prompt.py` reads `extras["station"]` unchecked). A caller can inject arbitrary scenario/system-prompt content and drive sessions for scenarios they were not assigned. The workers never score, so mark integrity is not directly affected — but content and cost are.

REMEDIATION

Rebuild the station server-side from the database using an identifier the caller is authorised for; treat `extras` as opaque hints only. Cap session creation per user.

SEC-008 No cap on voice session length → LLM/voice cost abuse

Fixed

CVSS 3.1 5.3 · AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:L | **OWASP** LLM10:2025

Nothing caps how long an assistant session runs; it continues until the user leaves, and the voice token TTL is one hour. The pacing clock deliberately never ends an interaction, and only the GPT-Live runtime carried a policy of its own (300 s); the default pipeline runtime ran until the client disconnected, so a single user could hold sessions open to run up unbounded LLM, speech-to-text and text-to-speech spend (denial-of-wallet).

REMEDIATION (APPLIED)

Added a platform-agnostic wall-clock cap (`livekit_assistant_worker/session_cap.py`): `register_session_cap` installs one timer in the session entrypoint for every runtime, and `SessionCap` shuts the job down (`ctx.shutdown(reason="session_cap")`) once the limit passes. It is a pure resource guard — no scoring, no content, no view of the interaction — so it respects the "LiveKit layer stays platform-agnostic" rule. Defaults to one hour (the token TTL), tunable via `ASSISTANT_MAX_SESSION_SECONDS`, disabled with `0` / `off`; GPT-Live's own 300 s policy still fires first. Test: `tests/test_assistant_worker_session_cap.py`. Per-user concurrent-session and daily-minute caps remain a backend backlog item.

SEC-009 Container runs as root; build-ARG token in image history

Fixed

CVSS 3.1 5.9 · AV:L/AC:H/PR:L/UI:N/S:C/C:H/I:L/A:N | **CIS** Docker 4.1 / 4.10 · CWE-522

The platform-api image defined no non-root `USER`, so a process compromise started as root in the container. `ENGINE_REPO_TOKEN` passed as a build ARG is persisted in image layer metadata (`docker history`) and readable by anyone who can pull the image. The repo also had **no `.dockerignore`**, so the whole context (any local `.env`, `serviceAccountKey.json`, keys) was sent to the builder and `COPY . .`'d in.

REMEDIATION (APPLIED)

The Dockerfile now reads the engine token from a **BuildKit secret** (`--mount=type=secret,id=engine_token`, with `syntax=docker/dockerfile:1.7`) — written, used and cleaned up in one layer, so it never lands in `docker history`; the build ARG survives only as a documented migration fallback. It adds a non-root user (`useradd --uid 10001 appuser`, `USER appuser`). A new `.dockerignore` excludes `.git`, `.env*`, `*.pem` / `*.key`, `serviceAccountKey.json`, `docs` and `tests`.

> **Deploy verification (supervised):** Railway must supply `ENGINE_REPO_TOKEN` as a build secret with id `engine_token`; the first build after this change needs a supervised check. The token should still be rotated once (tracked under SEC-012).

SEC-011 Password reset on SSO-only accounts

Mitigated — corrected in v1.1

CVSS 3.1 ~2.0 (was 5.4) | **OWASP** API2:2023 · WSTG-ATHN-09

The v1.0 draft rated this Medium/Open. Direct code verification showed the risk is **already blocked**: `begin_password_reset` returns `None` (no token, no email) when `password_login_disabled` is set, and `confirm_password_reset` re-checks the same flag as defence in depth and raises `InvalidResetToken` — so even a token minted by another path (e.g. invitations) cannot set a password on an SSO-only account.

RESIDUAL (BACKLOG, COSMETIC)

`service_invitations.py` still *mints* a reset token for an SSO-only invitee without the early check; the token is inert (the confirm guard rejects it), so this is tidiness, not a vulnerability. Add the flag check at mint time to avoid issuing a dead token.

SEC-005 Missing security headers; schema exposed when deployed

Fixed

CVSS 3.1 3.7 · AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N | **OWASP** A05:2021 · WSTG-CONF-07

The API returned no baseline security headers, and `/openapi.json` — the full route inventory — was served even when deployed (only `docs_url` was gated), aiding reconnaissance.

REMEDIATION (APPLIED)

Added `SecurityHeadersMiddleware` (nosniff, frame-deny, no-referrer, a strict CSP for the JSON API, Permissions-Policy, COOP/CORP, and HSTS when deployed) and gated `openapi_url` off when deployed. Added `Permissions-Policy` to the web app. Test: `tests/security/test_security_headers.py` — headers present; schema and docs both `404` under a deployed build.

SEC-006 No enforcing Content-Security-Policy on the web app

Partial

CVSS 3.1 4.7 · AV:N/AC:H/PR:L/UI:R/S:C/C:L/I:L/A:N | **OWASP** A05:2021

The web app ships no CSP — the primary defence-in-depth control that would *contain* an XSS such as SEC-002 even if a sink were reintroduced. `Permissions-Policy` was added; the enforcing CSP is deliberately deferred because a Next.js CSP needs per-request script nonces and correct `connect-src` for the API and LiveKit websockets, and a wrong CSP breaks the app silently.

REMEDIATION

Introduce a nonce-based CSP in `Report-Only` mode on staging, tune against real violation reports through the full user journey, then enforce.

SEC-010 Worker pods lack securityContext / NetworkPolicy

Fixed

CVSS 3.1 4.0 · AV:L/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:L | **CIS** Kubernetes 5.x

Worker pods defined no `securityContext` (no `runAsNonRoot`, dropped capabilities, or `allowPrivilegeEscalation: false`), no `NetworkPolicy`, a mutable `:production` tag and no CPU limit; `.dockerignore` let `runs/` and `dist/` into the image.

REMEDIATION (APPLIED)

Both worker deployments now carry a pod `securityContext` (`runAsNonRoot: true`, `runAsUser/Group: 10001`, `fsGroup: 10001`, `seccompProfile: RuntimeDefault`) and a container `securityContext` (`allowPrivilegeEscalation: false`, `capabilities: drop: [ALL]`). A new `k8s/networkpolicy.yaml` confines inbound traffic to the

probe/agent ports (8080/8087/8088) and denies everything else in; `.dockerignore` now excludes `runs/`, `dist/`, `*.pem`, `serviceAccountKey.json` and `KMS`. Test: `tests/test_k8s_config.py` (unprivileged + drop-capabilities; NetworkPolicy port confinement).

`readOnlyRootFilesystem` is left `false` (the worker caches plugin models under `/app/.cache`), CPU limits stay off for real-time-audio latency, and pinning image digests over `:production` is a deploy-workflow change — all documented as by-design/backlog. Egress is intentionally unrestricted (workers dial LiveKit Cloud, backends and LLM providers on non-fixed addresses); the inbound lockdown is the win, pending a supervised `kubectl apply`.

SEC-013 Station photo upload validates extension only

Backlog

CVSS 3.1 3.1 · AV:N/AC:H/PR:L/UI:R/S:C/C:L/I:L/A:N | **OWASP** WSTG-BUSLOGIC-09 · CWE-434

Upload validation is otherwise sound — size is capped at 10 MB and checked server-side, and the storage key is a fresh `uuid4` so filename path-traversal is impossible. But content type is validated by *file extension only*; there is no magic-byte sniff. A `.png` containing arbitrary bytes passes. Mitigated (not eliminated) by S3 storing the object as `binary/octet-stream`, which browsers download rather than render.

REMEDIATION

Sniff real content (e.g. `PIL.Image.verify()`); set explicit safe `Content-Type` / `Content-Disposition`; ensure `nosniff` at the edge; prefer presigned URLs over a public bucket.

SEC-014 Org may enrol an un-rostered arbitrary user id into a circuit

Fixed

CVSS 3.1 3.5 · AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:N | **OWASP** API1:2023 · CWE-639

This is *not* a cross-org read IDOR — the `trainee_id` concern was tested and is not present (see positive observations). But `roster/service.py::ensure_rosterable` deliberately did not refuse a `users.id` that is not on the org's roster, so an org admin could place an arbitrary, enumerated account id into their own org's circuit without that user's consent. It discloses no data but attaches a real account to an examination it never joined.

REMEDIATION (APPLIED)

Owner decision (25 Sep): not intended — locked down. `ensure_rosterable` already raised `TraineeNotOnRosterError` for a `users.id` with no roster link, but the circuit-enrolment routes did not declare it, so the `UndeclaredErrorCode` middleware turned it into a 500 and the route read as permissive. The three routes (`add_circuit`, `rebuild_circuit`, `append_trainees`) now declare that error, so an un-rostered id is refused with a clean `409 trainee_not_on_roster`. Test: `tests/http/test_sittings_routes.py` (an un-rostered or revoked user id is refused, not enrolled).

VERIFICATION ROUND · 25 SEP 2026 — DEEP-DIVE FINDINGS

SEC-015 Entra SSO: no OAuth `state`, no OIDC `nonce`

Fixed

CVSS 3.1 5.4 · AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N | **OWASP** A07:2021 · WSTG-SESS-05 · OIDC Cc

The Entra sign-in URL (`sso/entra.py::sign_in_url`) carried no `state` parameter, and the callback validated the id_token signature, `aud`, `iss` and `exp` — good — but never bound a `nonce`. This is classic OAuth *login-CSRF* / session fixation: an attacker who completes their own Entra login can get a victim's browser to submit the attacker's authorization `code`, signing the victim into the **attacker's** account. Because the platform sets no session cookie and returns tokens in the JSON body, impact is capped at "victim signed into the attacker's account" rather than token theft — but it is a real, standard gap. (The Clerk flow verifies a session token against Clerk's API and is not raw OIDC, so `state` / `nonce` do not apply there the same way.)

REMEDIATION (APPLIED)

Implemented the stateless signed-`state` design that fits the token-in-JSON architecture (no server session store). A new `sso/state.py` mints a short-lived (600 s) HS256 token carrying the initiating org slug and a random `nonce`, purpose-tagged `sso-state`; `build_login` issues it and passes both `state` and `nonce` to `sign_in_url`, and `complete_callback` requires a valid state for Entra, recovers the expected `nonce`, and `entra.verify` refuses an id_token whose `nonce` does not match. Clerk is unaffected (Entra-only). Tests: `tests/security/test_sso_state.py` and `tests/http/test_sso_login_routes.py` (a callback with no state or a foreign org's state is refused with `422 sso_verification_failed`).

SEC-018 Slide renderer path traversal via LLM-generated image URLs

Fixed

CVSS 3.1 5.0 · AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:N/A:N | **OWASP** WSTG-ATHZ-01 · CWE-22

`slide_renderer.py::_resolve_image_path` joined an `image_url` onto a base directory with `Path.__truediv__`, which does not neutralise `..`. The `image_urls` are **LLM-generated** (produced by the slide-generation prompt), so a prompt-injection from station/scenario content could redirect the field. A crafted `../../../../etc/<image>` escaped the slides tree, and any image-parseable file readable by the worker was embedded into the rendered video (then uploaded to S3 and downloadable) — a filesystem-read primitive. The trigger route is `PlatformAdmin`-gated, hence Medium.

REMEDIATION (APPLIED)

`_resolve_image_path` now rejects absolute paths and any `..` component, resolves the candidate and asserts containment within its base (`Path.relative_to`); an unsafe URL resolves to a guaranteed-absent path so every caller's existing `.exists()` guard skips it. Behaviour is unchanged for legitimate images. Test: `tests/security/test_slide_image_path_traversal.py` (4 cases).

SEC-019 Clerk session id spliced unescaped into outbound API URL

Fixed

CVSS 3.1 3.1 · AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N | **CWE** 88 (URL/argument injection)

`sso/clerk.py::verify` reads the `sid` claim from a JWT *before* its signature is checked, then spliced `session_id` unescaped into `{CLERK_API_BASE}/sessions/{session_id}/verify`. A `/`, `?`, `#` or `..` in the attacker-influenced `sid` could reshape the URL that the org's Clerk secret key is attached to. The host is a hard-coded literal, so this is not full SSRF, but untrusted input was building a privileged request URL.

REMEDIATION (APPLIED)

`session_id` and `user_id` are now `quote(…, safe="")`-encoded before interpolation, matching the discipline already used for `redirect_url`. Test: `tests/security/test_clerk_session_id_url_encoding.py` — a crafted `sid` is percent-encoded and the raw traversal path never appears.

SEC-016 Stream/voice ticket is time-bound but not single-use

Fixed

CVSS 3.1 2.6 · AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N | **CWE** 294 · WSTG-SESS-03

The SSE stream ticket (`core/streams.py`) is a stateless HS256 JWT with a 60-second TTL. `decode_stream_ticket` validated signature/iss/exp/typ only — there was no `jti` or server-side consumption tracking, so the same ticket could open multiple concurrent

`EventSource` connections or be replayed within its window. Low: bounded by the short TTL and by the ticket granting only read access the holder was already entitled to.

REMEDIATION (APPLIED)

`issue_stream_ticket` now stamps each ticket with a random `jti`, and redemption claims it via Redis `SET NX EX` (keyed `stream-ticket-used:<jti>`, TTL = the ticket's remaining life): the first redemption wins, a replay is refused. The guard **fails open** — a ticket with no `jti`, or a Redis that is off or erroring, redeems as before — a better trade than dropping every live stream during a cache blip (documented). Test: `tests/security/test_stream_ticket_single_use.py` (redeem-once-then-replay-refused; two distinct tickets each redeem; fails open when Redis is off or errors).

SEC-017 Stripe checkout omits the `metadata.price_id` its webhook path depends on

Info · reliability

Class Reliability (security-adjacent) · fails closed | **Where** billing/checkout.py vs billing/

Recorded while verifying payment integrity, and masked by the tests. `StripeCheckout.create()` never sets any `metadata` field and never requests `expand[]=line_items`, yet `_price_id()` reads `metadata.price_id` (falling back to expanded line items). A real `checkout.session.completed` webhook likely arrives with neither, so `_apply_checkout` raises `InvalidWebhook(... lacks a price ...)` for genuine payments. It **fails closed** (no wrongful grant; the ledger row persists for reconciliation), so it is not attacker-exploitable — but a paying customer might not be granted access. The billing tests fabricate `metadata.price_id`, so the gap is not caught.

REMEDIATION

Set `metadata[price_id]` (or `payment_intent_data[metadata]`) at checkout-session creation, or expand `line_items`; add an end-to-end test against a real Stripe test-mode webhook rather than a hand-built event.

SEC-020 Community `set_like` skips an audience visibility check

Info

Class Informational | **OWASP** API1:2023 · CWE-639 | **Where** community/service.py

Unlike `question_detail` / `list_replies` (which re-check `can_see(audience_of(row), viewer_audiences)` and return `PostNotFound` for out-of-audience content), `set_like` toggles a like without an audience check. A user who guesses a post id could like/unlike content outside their visibility scope. No data is disclosed — only an engagement counter moves — so this is cosmetic.

REMEDIATION

Apply the same `can_see` audience check before toggling a like.

---

05 Positive observations

Controls that held up under review — as important as the findings for an auditor, because they show the platform was built with security in mind.

Robust token verification. `HS256` is pinned (an `alg:none` or wrong-secret token is refused), the issuer, `exp`, `iat` and `sub` are required, and `typ == "access"` is checked so a refresh token cannot open an access route. Proven by 5 passing negative-control tests.

Strong password storage & session hygiene. Argon2 hashing; refresh tokens stored hashed and rotated, with reuse detection that revokes the chain.

Fail-closed worker auth. `require_worker_secret` uses `hmac.compare_digest` and refuses when the secret is unset or mismatched.

No cross-org `trainee_id` IDOR. Despite `trainee_id` meaning two different things, every endpoint filters by the caller's server-derived `org_id`; candidate self-service derives ids from the authenticated user. Supplying a foreign-org id returns nothing.

Tenant suspension is enforced live. Organisation status is re-read from the database each request, so a suspended tenant loses access within the token's 15-minute life.

Receipts are HTML-escaped. Every user-influenced field passes through `html.escape` before interpolation; no injection path found.

Parameterised SQL, confined to repositories. The only raw `text()` queries bind every user value as a parameter; a SAST hit here was confirmed a false positive. Architecture guard tests keep SQL inside repositories.

Clean dependency & secret posture on new code. `npm audit`: 0 vulnerabilities. Gitleaks on assessment-platform: clean.

Server-authoritative authorization in the web app. Permissions come from `/me/capabilities`, never a role in the browser; the exam clock is the server's.

Whole injection classes are absent. A deliberate sweep for SSRF, insecure deserialization, server-side template injection, OS-command injection, XXE and ReDoS found none reachable from untrusted input — no `pickle` / `yaml.load` / `eval` on request data, no user-controlled outbound URL, no template render over user strings, no `subprocess` with request data, no exposed XML parse, no catastrophic regex.

Mass-assignment is structurally prevented. Endpoints bind typed Pydantic models with explicit fields, so a client cannot smuggle `is_platform_admin`, `org_id`, `role` or `status` into an update — unknown keys are dropped and privileged fields are server-set.

Payment integrity is sound. The Stripe webhook verifies the signature in constant time and *fails closed*, rejects the wrong event type, is idempotent against replay, and grants entitlements only after a verified `checkout.session.completed`. Entitlement reads filter on `live` and fail closed.

Voice-token authorization is least-privilege. Voice, observer and join tokens are minted only after an ownership check; the participant identity is server-assigned (no candidate can claim `agent-*`); and the observer grant is subscribe-only with no publish rights.

---

06 Remediation roadmap

PRIORITY	FINDINGS	ACTION	TARGET
Done	001, 002, 005, 018, 019	Fixed during engagement (regression tests green)	Complete
Done	008, 009, 010, 014, 015, 016	Remediation pass (25 Sep): voice session-length cap; container non-root + BuildKit secret + `.dockerignore`; worker pod	Complete

PRIORITY	FINDINGS	ACTION	TARGET
		securityContext + NetworkPolicy; circuit-enrolment roster lock-down; Entra `state` / `nonce`; stream-ticket single-use — all with regression tests	
Immediate	012	Rotate all exposed secrets; move key out of tree (owner action)	≤ 1 week
Deploy verify	009, 010	First Railway build with the `engine_token` BuildKit secret; `kubectl apply` the hardened worker manifests + NetworkPolicy and smoke a session	Next deploy
Short term	003, 004, 007	Rate-limit hardening (XFF + fail-closed); validate assistant `extras` server-side; per-user session/rate caps	2–4 weeks
Short term	006	Nonce CSP in Report-Only on staging, then enforce	2–4 weeks
Medium term	013	Upload content sniffing + safe `Content-Type` / `Content-Disposition`	4–8 weeks
Housekeeping	011, 017, 020	Mint-time SSO guard; set Stripe `metadata.price_id`; audience check on community like	Backlog
Follow-on	all	Production engagement (v2): TLS, headers, CORS, rate limits at the live edge; authenticated DAST	Next round

07 Appendices

A · Coverage checklist

AREA	RESULT	NOTES
Schema exposure	Fixed	/openapi.json gated when deployed
Config & headers / CORS	Fixed	Headers added; CORS origins allow-list; re-verify in v2
Authentication	Pass	Token verification hardened; argon2; SSO-reset blocked in code (011)
Session management	Pass	Refresh rotation + reuse detection; stream ticket time-bound (016)
Authorization (BOLA/BFLA)	Fixed	Escalation fixed (001); IDOR tested — not present; BFLA holds; 014 noted
Mass assignment	Pass	Typed request models drop unknown / privileged keys
SSO (OAuth / OIDC)	Finding	Clerk verify hardened (019 fixed); Entra lacks state/nonce (015)
Rate limiting / resource use	Finding	XFF + fail-open (003); session caps (008)
Injection (SQLi)	Pass	Parameterised; SAST hit false positive
SSRF / deserialization / SSTI / cmd / XXE / ReDoS	Absent	Swept in the verification round — no untrusted sink found
Cross-site scripting	Fixed	Markdown sanitised (002)
Path traversal	Fixed	LLM image-URL traversal fixed (018); upload keys are uuid4
Client-side (CSP)	Partial	Enforcing CSP deferred to staging (006)
Payments & entitlements	Pass	Webhook signature verified, fails closed, idempotent; gates filter on `live` ; price_id reliability note (017); deeper tests in v2
LLM / voice	Finding	Output handling & traversal fixed (002, 018); prompt/cost open (007, 008)
Worker / dispatch auth	Pass	X-Worker-Secret fail-closed
Voice-token authorization	Pass	Ownership-checked; server identity; observer subscribe-only
File upload	Finding	Size/traversal safe; content-type extension-only (013)
Secret management	Finding	Secrets in history/tree (012)

AREA	RESULT	NOTES
Container & Kubernetes	Finding	Root container, build-ARG token, pod hardening (009, 010)

B · Role × route access matrix (result)

ACTOR	ACTION	EXPECTED	RESULT
`partner_admin`	grant institution_admin / owner	403	Enforced
any org admin	write an undefined role string	422	Enforced
`platform_admin`	membership admin via console	allowed (known roles)	Enforced
org A member	read org B via foreign trainee_id	no data	Verified
`trainee`	run / advance a circuit	403	Enforced
org admin	call a `/platform/*` route	403	Enforced
any client	smuggle `is_platform_admin` in a body	ignored	Enforced
candidate	join a voice room as `agent-*`	rejected	Enforced
observer	publish audio / data on a session	rejected	Enforced
forged / alg:none token	any authenticated route	401	Enforced
refresh token as access token	access route	401	Enforced

C · Tools & tests

Trivy 0.58.1 · Gitleaks 8.21.2 · Semgrep 1.99.0 · npm audit (npm 10 / Node 22) · pytest + httpx (Python 3.11) · node --test (Node 22).

Regression tests added: `tests/security/test_auth_tokens.py`, `test_membership_role_escalation.py`, `test_security_headers.py`, `test_slide_image_path_traversal.py` (SEC-018), `test_clerk_session_id_url_encoding.py` (SEC-019); `platform-web/.../renderMarkdown.test.ts`. Full run at report time (incl. the verification round): `tests/security` **15 passed**; the sso/media/clerk/slide modules touched by the new fixes

431 passed with zero regressions; `mypy` and `ruff` clean; web markdown suite 7 passed; `tsc --noEmit` clean.
